

Algoritmos de Ordenação Não Recursivos: Uma Análise Comparativa entre SelectionSort, InsertionSort e BubbleSort

1. Introdução

A ordenação de dados é um dos problemas mais clássicos e fundamentais da Ciência da Computação. Ordenar uma sequência de elementos significa rearranjá-los em uma determinada ordem (geralmente crescente ou decrescente). Essa tarefa é essencial para otimizar outros algoritmos, como a busca binária.

Os algoritmos de ordenação podem ser divididos entre **recursivos** (como Quicksort e Mergesort) e **não recursivos/iterativos** (como SelectionSort, InsertionSort e BubbleSort). Este trabalho tem como objetivo analisar o funcionamento, a implementação e a eficiência dos três algoritmos iterativos mais tradicionais, conhecidos por sua simplicidade de implementação.

2. Conceitos Fundamentais

Antes de analisar cada método, é importante definir duas métricas usadas para comparar algoritmos:

1. **Complexidade de Tempo (Notação Big-O):** Mede o crescimento do tempo de execução à medida que o número de elementos (N) aumenta. Avalia-se o *Melhor Caso*, o *Caso Médio* e o *Pior Caso*.
2. **Estabilidade:** Um algoritmo é considerado **estável** se ele mantém a ordem relativa de elementos que possuem chaves/valores iguais.

3. Os Algoritmos de Ordenação

3.1. BubbleSort (Ordenação por Bolha)

Como funciona?

O **BubbleSort** percorre o vetor várias vezes. A cada passagem, ele compara pares de elementos adjacentes e os troca de lugar se estiverem fora de ordem. Dessa forma, os maiores elementos "flutuam" para o final do vetor, assim como bolhas de ar sobem em um líquido.

Algoritmo em C++:

```
void bubbleSort(vector<int>& vec) {
```

```

    size_t n = vec.size();
    for (size_t i = 0; i < n - 1; ++i) {
        bool trocou = false; // Otimização para detectar vetor já
ordenado
        for (size_t j = 0; j < n - i - 1; ++j) {
            if (vec[j] > vec[j + 1]) {
                swap(vec[j], vec[j + 1]);
                trocou = true;
            }
        }
        // Se nenhuma troca ocorreu nesta passagem, o vetor já
está ordenado
        if (!trocou) break;
    }
}

```

- **Vantagens:** Extremamente simples de entender e codificar.
- **Desvantagens:** É o mais ineficiente dos três para grandes volumes de dados.

3.2. SelectionSort (Ordenação por Seleção)

Como funciona?

O **SelectionSort** divide o vetor em duas partes: a sublista de elementos já ordenados e a sublista de elementos não ordenados. Em cada passo, ele procura o **menor elemento** presente na parte não ordenada e o troca com o elemento que está no início dessa parte não ordenada.

Algoritmo em C++:

```

void selectionSort(std::vector<int>& vec) {
    size_t n = vec.size();
    for (size_t i = 0; i < n - 1; ++i) {
        size_t minIdx = i;
        for (size_t j = i + 1; j < n; ++j) {
            if (vec[j] < vec[minIdx]) {
                minIdx = j; // Atualiza o índice do menor elemento
            }
        }
        if (minIdx != i) {
            swap(vec[i], vec[minIdx]);
        }
    }
}

```

- **Vantagens:** Realiza um número mínimo de trocas ($O(N)$ trocas no máximo), o que é útil quando a escrita na memória é uma operação custosa.

- **Desvantagens:** Mesmo se o vetor já estiver ordenado, ele continuará fazendo todas as comparações até o fim.

3.3. InsertionSort (Ordenação por Inserção)

Como funciona?

O **InsertionSort** funciona de maneira semelhante à forma como organizamos cartas em um jogo de baralho. Ele constrói o vetor ordenado um elemento por vez. Pega-se o elemento atual e o compara com os elementos à sua esquerda, "empurrando" os maiores para a direita até encontrar a posição correta para **inseri-lo**.

Algoritmo em C++:

```
void selectionSort(std::vector<int>& vec) {
    size_t n = vec.size();
    for (size_t i = 0; i < n - 1; ++i) {
        size_t minIdx = i;
        for (size_t j = i + 1; j < n; ++j) {
            if (vec[j] < vec[minIdx]) {
                minIdx = j; // Atualiza o índice do menor elemento
            }
        }
        if (minIdx != i) {
            swap(vec[i], vec[minIdx]);
        }
    }
}
```

- **Vantagens:** É extremamente eficiente para vetores pequenos ou vetores que já estão quase ordenados.
- **Desvantagens:** Em vetores inversamente ordenados, realiza um número elevado de movimentações de memória.

4. Tabela Comparativa

Algoritmo	Complexidade (Melhor Caso)	Complexidade (Caso Médio)	Complexidade (Pior Caso)	Estável ?	Trocas de Memória
BubbleSort	$O(N^2)$ (com otimização)	$O(N^2)$	$O(N^2)$	Sim	Alto

SelectionSort	$O(N^2)$	$O(N^2)$	$O(N^2)$	Não	Baixo ($O(N^2)$)
InsertionSort	$O(N)$	$O(N^2)$	$O(N^2)$	Sim	Médio

5. Conclusão

Embora todos os três métodos possuam uma complexidade de tempo de pior caso igual a $O(N^2)$ — o que os torna inviáveis para grandes bases de dados em comparação a algoritmos como o *Quicksort* ($O(N \log N)$) —, cada um possui suas particularidades:

1. O **BubbleSort** é útil primordialmente para fins didáticos e para ensinar o conceito de trocas e laços aninhados.
2. O **SelectionSort** destaca-se caso o custo de mover/escrever dados na memória seja crítico, pois minimiza a quantidade de trocas.
3. O **InsertionSort** é o melhor entre os três na prática, sendo inclusive utilizado como base em algoritmos híbridos reais (como o *Timsort* em Python e Java) para subvetores pequenos ou parcialmente ordenados.

6. Referências Bibliográficas

CORMEN, Thomas H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012. Acesso: 15 de Ago. de 2026.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. **Estruturas de dados usando C**. São Paulo: Pearson, 1995. Acesso: 16 de Ago. de 2026.

ZIVIANI, Nivio. **Projeto de algoritmos: com implementações em Pascal e C**. 3. ed. São Paulo: Cengage Learning, 2011. Acesso: 17 de Ago. de 2026.
